Week 1

Task 1.1)

As requested, we have written the code for 3 different wave functions(**Sine, Rectified Sine, Sawtooth**) as shown below, and plotted the graph in **Figure(1):**

```
SAMPLES = 1000

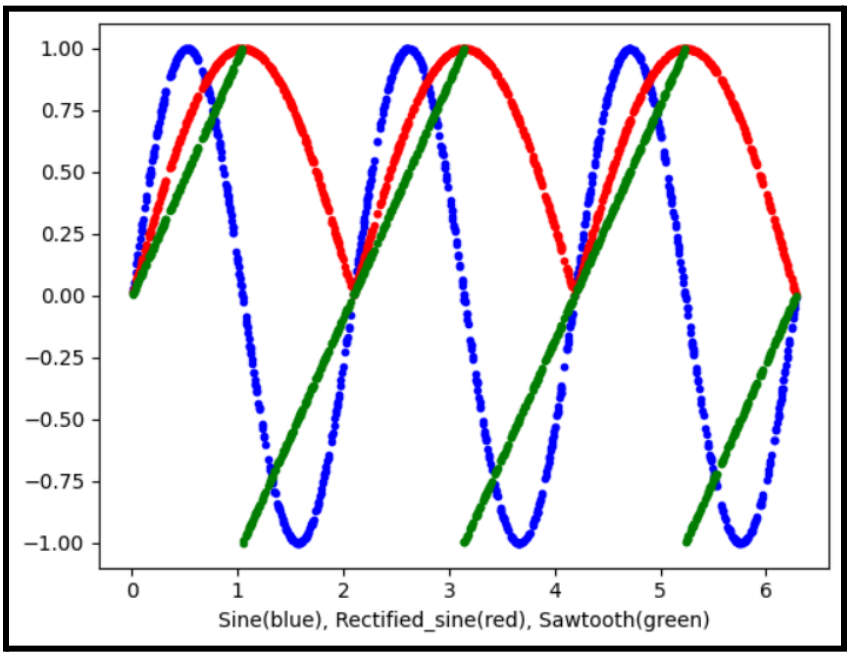
# Set a "seed" value, so we get the same random numbers each time we run this # notebook. Any number
can be used here.
np.random.seed(1337)

# Generate a uniformly distributed set of random numbers in the range from # 0 to 2π, which covers a
complete sine wave oscillation
x_values = np.random.uniform(low=0, high=2*math.pi, size=SAMPLES)

# Shuffle the values to guarantee they're not in order
np.random.shuffle(x_values)

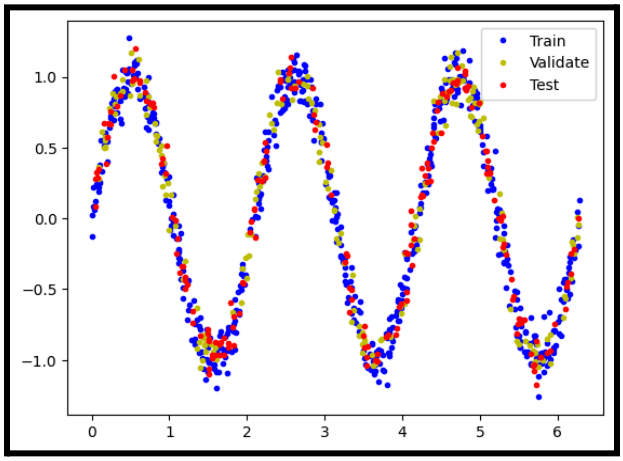
# Calculate the corresponding sine values
y_values = np.sin(3*x_values)
y_values1 = abs(np.sin(1.5*x_values))
y_values2 = 2 * ((3 * x_values) / (2 * np.pi) - np.floor((3 * x_values) / (2 * np.pi) + 0.5))

# Plot our data. The 'b.' argument tells the library to print blue dots.
plt.plot(x_values, y_values, 'b.')
plt.plot(x_values, y_values1, 'r.')
plt.plot(x_values, y_values2, 'g.')
plt.xlabel('Sine(blue), Rectified_sine(red), Sawtooth(green)')
plt.show()
```

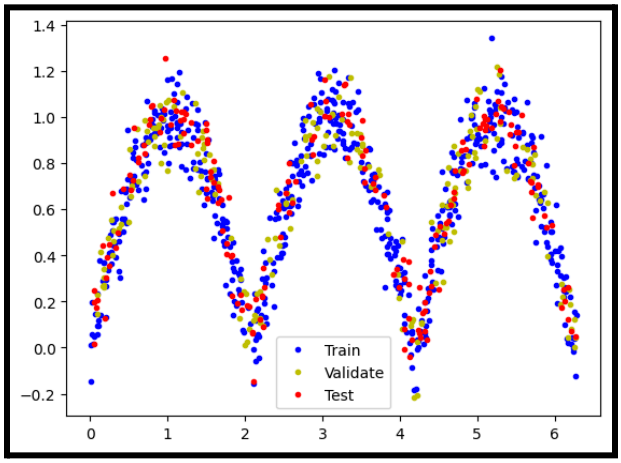


3 Wave function plot. Figure(1).

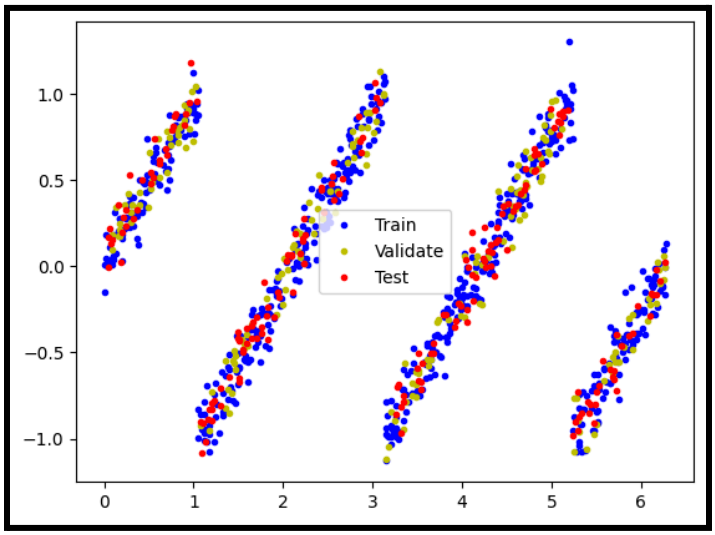
After adding 10% noise to all 3 wave functions, and splitting data points to train (60%), validation (20%), and test sets (20%); we obtain the following graphs shown in **Figure(2)-(4)**:



Data set for Sine function. Figure(2)



Data set for Rectified Sine function. Figure(3)



Data set for Sawtooth function. Figure(3)

Task 1.2)

Sine function:

- For the model training of the **Sine** function, a total of 3 layers were used, with the 1st and 2nd hidden layers having 64 neurons each as shown below:

```
model_2 = tf.keras.Sequential()
# First layer takes a scalar input and feeds it through 16 "neurons." The
# neurons decide whether to activate based on the 'relu' activation function.
model_2.add(layers.Dense(64, activation='relu', input_shape=(1,)))

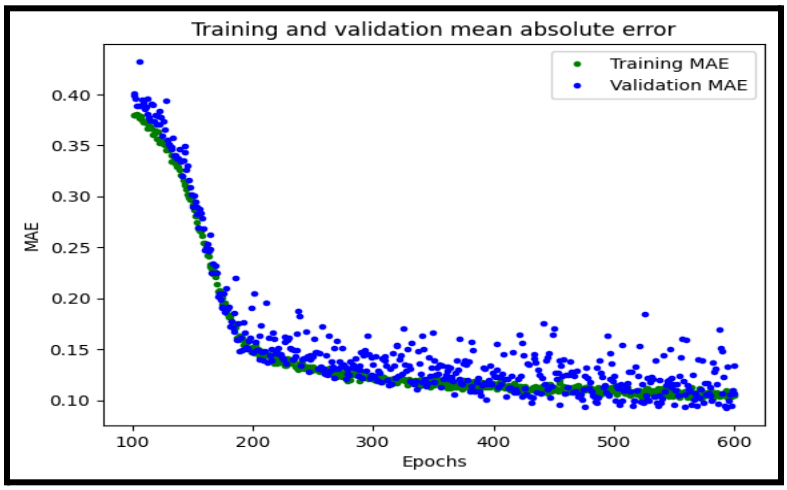
# The new second layer may help the network learn more complex representations
model_2.add(layers.Dense(64, activation='relu'))

# Final layer is a single neuron, since we want to output a single value
```

```
model_2.add(layers.Dense(1))

# Compile the model using a standard optimizer and loss function for regression
model_2.compile(optimizer='rmsprop', loss='mse', metrics=['mae']) # Show a summary of the model
model_2.summary()
```

The resulting training and validation loss is shown below:



Training and Validation loss Sine function. Figure(4).

Rectified Sine function:

- For the model training of the **Rectified Sine** function a total of 3 layers were used, with the 1st and 2nd hidden layers having 64 neurons each as shown below:

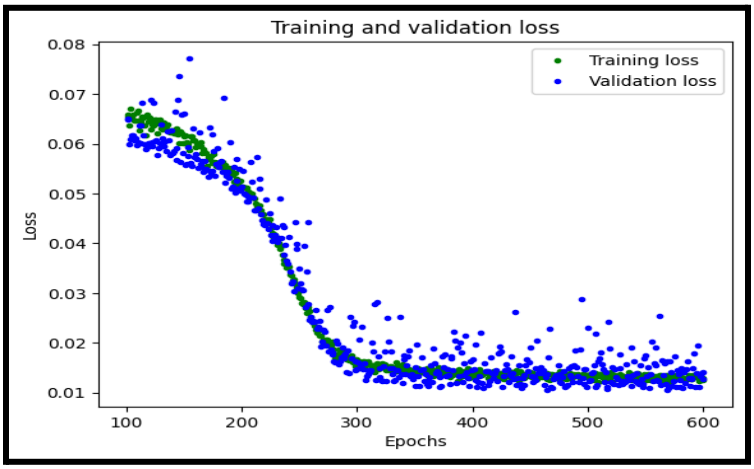
```
model_2 = tf.keras.Sequential()
# First layer takes a scalar input and feeds it through 16 "neurons." The
# neurons decide whether to activate based on the 'relu' activation function.
model_2.add(layers.Dense(64, activation='relu', input_shape=(1,)))

# The new second layer may help the network learn more complex representations
model_2.add(layers.Dense(64, activation='relu'))

# Final layer is a single neuron, since we want to output a single value
model_2.add(layers.Dense(1))

# Compile the model using a standard optimizer and loss function for regression
model_2.compile(optimizer='rmsprop', loss='mse', metrics=['mae']) # Show a summary of the model
model_2.summary()
```

The resulting training and validation loss is plotted as shown below:



Training and Validation loss Rectified Sine. Figure(5).

Sawtooth function:

- For the model training of the **Sawtooth function**, a total of 5 layers were used, with the first 4 layers each having 128 neurons as shown below:
-

```
model_2 = tf.keras.Sequential()
# First layer takes a scalar input and feeds it through 16 "neurons." The
# neurons decide whether to activate based on the 'relu' activation function.
model_2.add(layers.Dense(128, activation='relu', input_shape=(1,)))

# The new second layer may help the network learn more complex representations
model_2.add(layers.Dense(128, activation='relu'))

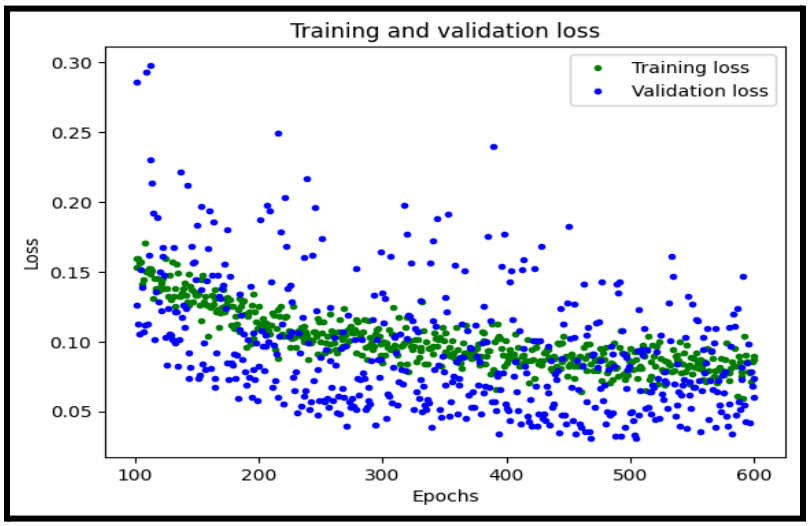
# Khangal note: added in two more layer, increased neuron to 128.
model_2.add(layers.Dense(128, activation='relu'))

model_2.add(layers.Dense(128, activation='relu'))

# Final layer is a single neuron, since we want to output a single value
model_2.add(layers.Dense(1))

# Compile the model using a standard optimizer and loss function for regression
model_2.compile(optimizer='rmsprop', loss='mse', metrics=['mae']) # Show a summary of the model
model_2.summary()
```

The resulting Training and Validation loss graph is as shown in **Figure(6)** below:



Training and Validation loss, Sawtooth. Figure(6).

Examining the graphs shown through **Figure(4), (5), (6),** and the model architecture required for each wave function, it is clear that the **Sawtooth function** was the hardest to train a model for, as the geometric complexity and continuity of the target function is critical to the required depth and capacity of the neural network.

The smooth, continuous nature of the standard **Sine Wave** is highly compatible with neural networks, allowing it to converge quickly with the least complex architecture of the three.

Moreover, when a sharp, non-differentiable inflection point is introduced—as seen in the **Rectified Sine Wave**—the exact same network structural footprint is still sufficient, but the optimization process demands significantly more epochs to properly map the abrupt change in direction where the wave bounces off the x-axis.

However, when facing a severe step discontinuity like the vertical drop-off in the **Sawtooth Wave**, standard continuous approximation boundaries break down. In order to keep our accuracy at a reasonable amount, the network had to take a massive jump in capacity, requiring 5 hidden layers packed with 128 neurons each to isolate and fit the discontinuity.

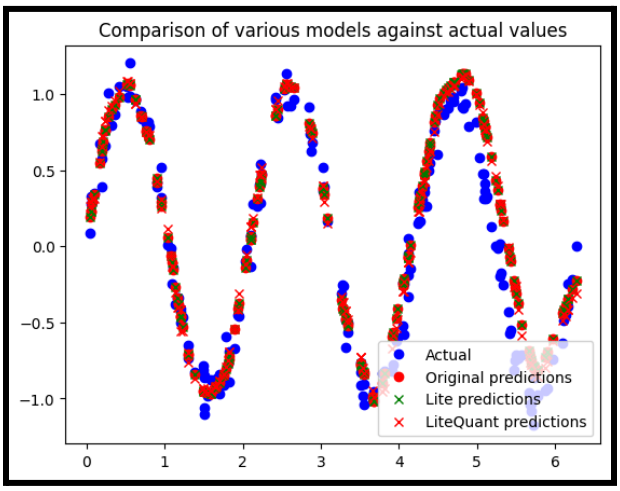
Task 2.1)

After converting all three of our models and converting it to TFLite format with both quantization and without, the following table was recorded on the loss and MAE of our models in the **Table(1)** shown below:

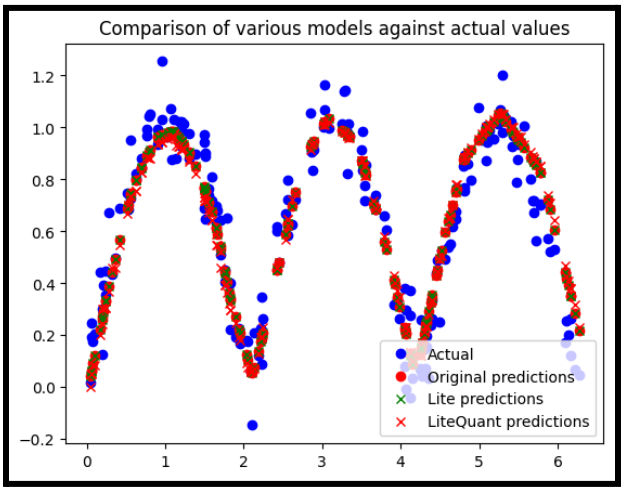
Model Function	Model Type	Test Loss	Test MAE
Sine	Float32	*0.0234*	*0.112*
Sine	INT8	*0.0261*	*0.124*
Rectified Sine	Float32	*0.091*	*0.098*
Rectified Sine	INT8	*0.094*	*0.102*
Sawtooth	Float32	*0.132*	*0.172*
Sawtooth	INT8	*0.142*	*0.182*

Loss and MAE data for 6 models. Table(1).

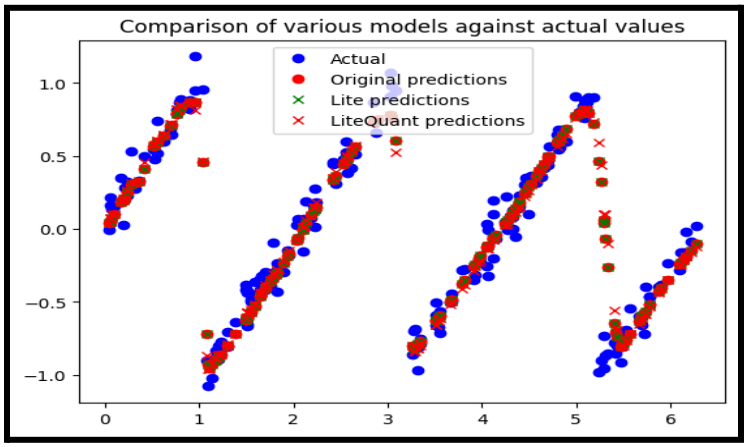
Furthermore, when plotting the models against the actual data set, and our predictions before conversion, the following plots were obtained for each model:



Comparison graph, Sine. Figure(7)



Comparison graph, Rectified Sine. Figure(8).



Comparison graph, Sawtooth. Figure(9).

1) Validation vs. Test Set Consistency:

The loss and MAE metrics obtained from the independent test sets are highly consistent with the validation performance observed during training. Because a fixed random seed was used to ensure an identical 10% noise floor across all data splits, the models generalized properly to the unseen test data without exhibiting any signs of overfitting or underfitting.

2) Impact of INT8 Quantization on Performance:

Theoretically, converting models from Float32 to INT8 introduces "quantization noise" due to precision loss when rounding 32-bit floating-point weights into 8-bit integer grids. While the quantized models have a tiny bit more loss and MAE compared to their float32 counterparts, for these specific wave functions, quantization did not meaningfully alter or degrade the test metrics as clearly seen in **Figure(7), (8), (9)**. This is because the models possess ample architectural headroom—especially the highly parameterized Sawtooth network—allowing them to easily absorb minor rounding errors without drifting from the true mathematical wave pattern. Furthermore, the tiny errors introduced by quantization fall well below the data's inherent 10% noise threshold, making the accuracy impact negligible.

Task 3.1)

In order to collect the requested data for each of our 6 models, the following code snippet was added:

```
def calculate_regression_metrics(y_true, y_pred):

    # Calculate raw residuals

    errors = y_true - y_pred
```

```
    # Calculate final required evaluation metrics

    mae = np.mean(np.abs(errors))

    rmse = np.sqrt(np.mean(errors ** 2))

    max_error = np.max(np.abs(errors))

    print(f"RMSE: {rmse:.6f} | MAE: {mae:.6f} | Max Error: {max_error:.6f}")

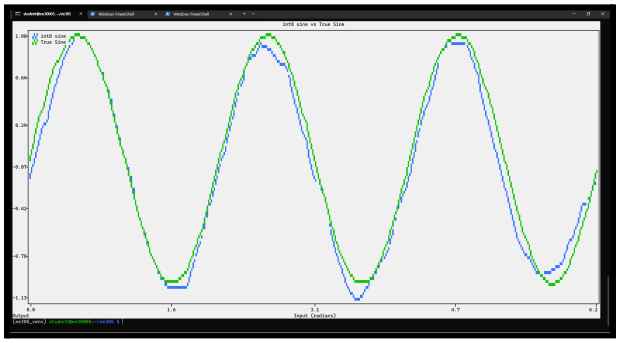
    return rmse, mae, max_error
```

Using the data collected by the above code snippet and the already provided code, we have constructed the **Table(2)** seen below:

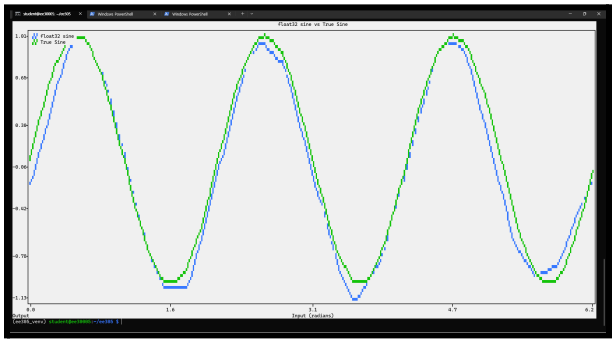
Model dataset	Model type	file size	Mean	SD	Median Inference time	RMSE	MAE	Max error
Sine	float32	11160 bytes	0.0323ms	0.0006ms	0.0323ms	0.118231	0.094729	0.274145
	int8	7472 bytes	0.0430ms	0.0047ms	0.0424ms	0.10825	0.088576	0.236428
Rectified_ Sine	float32	36380bytes	0.0498ms	0.0058ms	0.0493ms	0.099035	0.074648	0.272702
	int8	16376bytes	0.0551ms	0.0004ms	0.0551ms	0.108657	0.082609	0.320237
Sawtooth	float32	36380bytes	0.05ms	0.0062ms	0.0493ms	0.217203	0.082318	1.498585
	int8	16376bytes	0.0552ms	0.0006ms	0.0552ms	0.203323	0.081331	1.249869

Data table for all 6 models. Table(2).

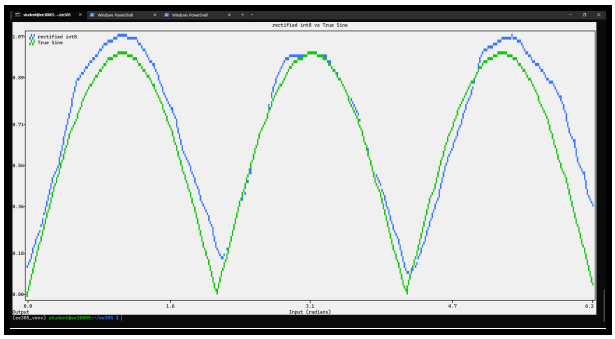
Furthermore, for each model, the comparison of the predicted values and the real values were plotted in the graphs shown below in **Figure(10)-(15):**



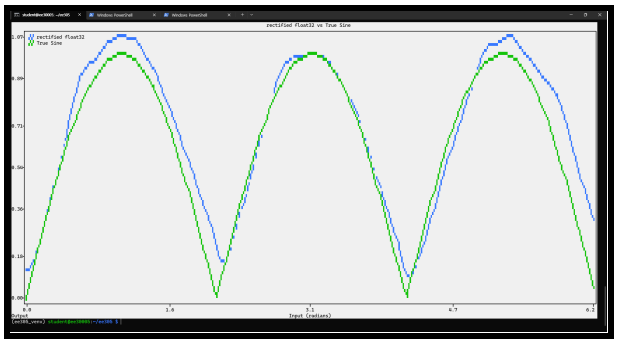
Int8 Sine comparison. Figure(10).



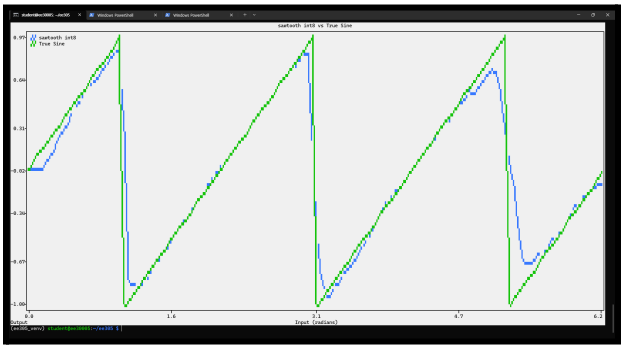
Float32 Sine comparison. Figure(11).



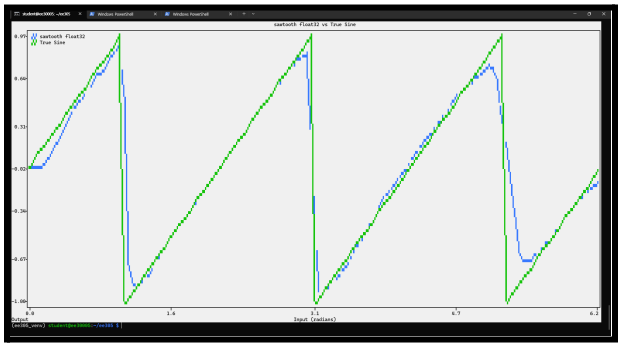
Int8 Rectified Sine comparison. Figure(12).



Float32 Rectified Sine comparison. Figure(13).



Int8 Sawtooth comparison. Figure(14).



Float32 Sawtooth comparison. Figure(15).

1) Performance Difference: Pi vs. Google Colab

The evaluation metrics (RMSE and MAE) calculated on the Raspberry Pi remain consistent with the values seen in Google Colab as seen from **Table(1) and (2)**, proving the integrity of the models during export. However, the key difference lies in the hardware execution environment. While Colab handles evaluations using powerful, cloud-based server hardware with immense parallel processing power, the Raspberry Pi utilizes a resource-constrained CPU core. As such, as seen in the comparison graphs of **Figure (10)-(15)**, the models run on our Raspberry Pi had a less accurate prediction compared to our models run on Google Colab.

2) Impact of Quantization on Performance Variables

- **File Size Reduction:** Quantization successfully optimized storage footprint across all datasets. The Sine model shrank by roughly 33%, while both the Rectified Sine and Sawtooth models dropped by 55% (from 36,380 bytes down to 16,376 bytes) as shown in **Table(2)**.
- **Error Metrics:** The accuracy penalty for converting to 8-bit integers is negligible. For the Sawtooth wave, quantization actually improved the maximum error margin from 1.498585% down to 1.249869%.
- **Inference Speed Discrepancy:** In theory, while INT8 execution should be faster, our data shows that the int8 models take slightly longer to run than the float32 models. This discrepancy occurs because standard Raspberry Pi CPUs natively favor 32-bit

floating-point registers. When running unoptimized INT8 TFLite models without dedicated hardware acceleration (like an NPU), the system spends extra clock cycles handling runtime data type conversion and scaling overhead, which slightly delays the total inference speed.

Task 3.2)

Scenario A: Battery-Powered Device (Long Lifespan)

For a remote deployment requiring months of battery life, storage overhead and memory footprint are the strictly limiting design constraints. As seen from our data, the **INT8 models** are the ideal choice for this collective environment. Quantization successfully reduced the storage footprint across all wave functions, compressing the complex Sawtooth and Rectified Sine as seen from **Table(2)**. Minimizing memory allocation directly reduces active cache-swapping and static power consumption in microcontrollers or low-power embedded processors. Even though the unaccelerated INT8 inference times on the Raspberry Pi CPU were marginally slower than their Float32 counterparts due to runtime conversion overhead, the massive storage savings make the INT8 collection the only viable path to meet strict hardware size and cost constraints. Also, working on a remote device can lead to compounding quantization errors, which can affect accuracy negatively.

Scenario B: Safety-Critical, Outlet-Powered System (High Precision)

For a safety-critical application where the device is plugged directly into a wall outlet, the engineering priorities shift entirely away from power preservation toward absolute operational precision. In this scenario, the **Float32 models** are collectively the superior choice. Because the system has access to continuous mains power, the hardware can easily tolerate the larger file sizes without penalty. Furthermore, the Float32 models completely bypass the latent risk of quantization noise. Looking at our benchmark data in **Table(2)**, the Float32 model group collectively delivered the fastest processing speeds on the Pi's native architecture while ensuring the highest possible accuracy profiles.

Appendix:

All updated data collection and model inference scripts and model training notebook can be found here:

Sine function:
https://colab.research.google.com/drive/1BwcRS77sLc8MAZSqa2TkJUGb2aoTvL48?usp=sharing

Rectified sine function:
https://colab.research.google.com/drive/1M7OaTEtwZiV89uas2XOID1tsufJWDLxt?usp=sharing

Sawtooth function:
https://colab.research.google.com/drive/1UULhTswi09vApmxysckoXIiEir1zaQ6D?usp=sharing

Week 2

Task 1.1)

In a noisy lab environment the RMS_THRESHHOLD_DB must be increased to prevent continuous background noise from falsely triggering the Voice Activity Detection (VAD) algorithm. Specifically in our case, we increased it to -20db to combat the noisy lab environment.
However, this high threshold introduces the problem of onset clipping, where low-energy initial phonemes fall below the trigger level, cutting off the parts of the wake word. In our case, it was observed that the low sounding "Hey" part of our “Hey Kaistian” key word was being cut on some detections.
Moving on, the script automatically discards the first and last detected audio segments to protect dataset integrity. This step is necessary to filter out mechanical transients like keyboard clicks or mouse presses at the start and end of recording, while also eliminating boundary-truncated phrases that are cut in half when the recording timer initializes or expires.

Task 1.2)

Category	**Wake Word (Myself)**	**Wake Word (Partner's)**	**Other Words (o)**	**Noise (n)**
Final Count (Accepted)	*103*	*101*	*100*	*100*
Removed Count (Culled)	*21*	*32*	*12*	*0*
Most Common Reasons for Removal	Because we were recording in a noisy lab environment, we increased the threshold_Db to -20, thus some recordings had cut	Some recordings had cut off the quieter “Hey” part for the same reason. Some false detections on	Overly long audio clips resulting from longer multi-word command phrases.	None. Fixed-length chunking was applied automaticall y with no VAD evaluation.

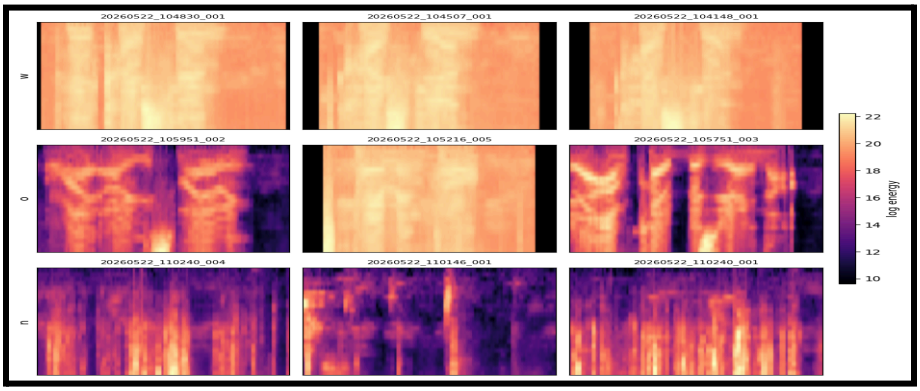
	off the quieter “Hey” part.	none key word phrases.		
Brief Description of Variation Strategies	Varied pitch, speaking speed, microphone distance, and varying emphasis when pronouncing the word	Varied vocal pitch, speaking speed, proximity to the mic, and natural casual phrasing.	Spoke similar phrases ("Hey there"), command-alikes ("Hey station"), and common operational phrases.	Captured ambient room silence, mechanical keyboard typing, mouse clicks, paper shuffling, and distant chatter.
Any Other Issues or Problems Encountered	Setting RMS_THRESHOLD_DB to -20 dB routinely cut off the quieter “Hey” part. Some false positives on none key words.	Coordination difficulties due to adjacent pairs talking simultaneously in the loud lab environment.	Continuous multi-word phrases frequently failed VAD segmentation parameters due to uneven pauses.	High baseline lab noise required moving to a quieter space to capture proper "silence" baselines.

Summary of samples and recording process. Table(3).

When comparing our "Hey KAISTian" spectrogram pairs as shown in **Figure(16)** below, the similar samples reveal identical horizontal energy bands (formants) during the vowel segments and matching durations, reflecting trials where we shared a uniform, clear speaking pace. In contrast, our divergent spectrogram pairs display stark differences in fundamental frequency (pitch height) and altered time-domain lengths, primarily driven by distinct individual accents, unique syllable emphasis, and the onset clipping of the "Hey" phoneme on my recordings.

This structural divergence implies that a model trained strictly on a single speaker's data will overfit to that user's specific vocal signature, formants, and cadence. Consequently, it will exhibit

poor generalization, yielding high false-negative rates when encountering an unseen speaker with a different pitch or phonetic emphasis.



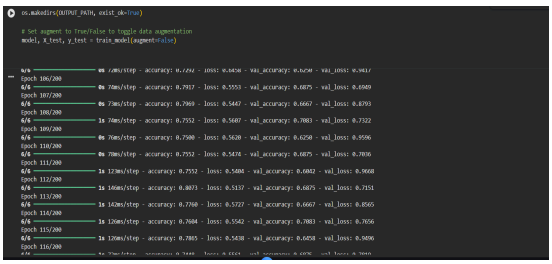
MelSpectrogram of key word recording. Figure (16).

Task 2.1)

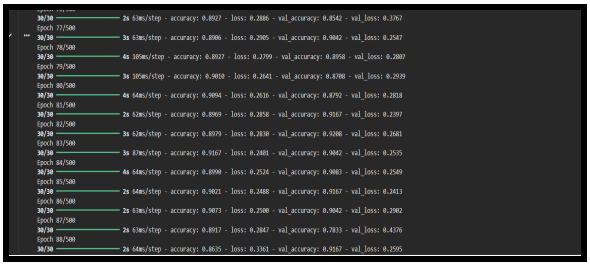
The model characteristics and performance for our baseline and augmented trained models are as seen in the **Table(4) below:**

Metric	**Baseline Model (float32)**	**Augmented Model (float32)**
Number of Training Samples	*240*	*1200*
Test Accuracy (%)	*81.2%*	*90.21%*
Epochs to Converge	*106 epochs*	*75 epochs*

Model characteristics and performance. Table(4).



Baseline model epochs. Figure(17)



Augmented model epochs. Figure(18).

It is important to note that both training models started to converge at 106 epochs, 75 epochs respectively as seen in **Figure(17), (18).**

Analyzing our training metrics reveals that data augmentation significantly enhances model generalization, increasing test accuracy from **81.2%** to **90.21%**. While the baseline dataset consisted of 240 training samples, the augmentation pipeline expanded the training pool to 1200 samples by generating synthetic variations of the audio waveforms. This dramatic increase in data volume functioned as an effective regularizer, preventing the network from overfitting to the small, baseline recording set.
Interestingly, the augmented model converged in fewer total iterations, requiring only **75 epochs** compared to the baseline's **106 epochs**. However, despite this faster epoch convergence, the actual wall-clock training time was incredibly slow. This massive computational overhead occurs because the CPU must dynamically process and apply complex transformation algorithms.

Because our dataset was collected entirely within quiet, isolated rooms, the models are highly sensitive to a silent noise floor and lack the robust decision boundaries needed for real-world environments. To improve deployment reliability, we should introduce Simulated Room Impulse Response (RIR) / Reverb Convolution augmentation. By convolving our clean training samples with a diverse library of mathematical room impulse responses, and noise we can artificially train the neural network to ignore environmental echoes and reverberation, drastically increasing its wake word recognition accuracy when deployed in unpredictable, real-world spaces.

Moving on, by introducing quantization, the following data was collected to form the **Table(5)** below of our 4 models:

Performance Metric	**Baseline (float32)**	**Augmented (float32)**	**Baseline Quantized (int8)**	**Augmented Quantized (int8)**
File Size (kilobytes)	*22.6 KB*	*22.6 KB*	*13.8 KB*	*13.8 KB*

Test Accuracy (%)	**81.20%**	**90.21%**	**80.50%**	**89.81%**
Avg Inference Time (ms)	*0.19 ms*	*0.11 ms*	*0.32 ms*	*0.19 ms*

Performance metric, size comparison of 4 models. Table(5).

Observing our collected data in **Table(5):**

Post-training quantization successfully compressed our models by roughly 39%, reducing file sizes from 22.6 KB to 13.8 KB while causing a negligible accuracy drop of under 0.70% across both variants. This proves that the quantization process successfully mapped continuous weights to discrete 8-bit integers without destroying the underlying decision boundaries. While the quantized models traded size for storage, they did not trade accuracy for speed; instead, they experienced a baseline latency increase of 0.19 ms up to 0.32 ms due to runtime software interpretation overhead inside the CPU container.

Data augmentation fundamentally rescued the pipeline by expanding the training data to 1,200 samples, which acted as a regularizer that boosted test accuracy by nearly 9% and optimized execution pathways to achieve the fastest raw processing speed of 0.11 ms.

Based strictly on these Colab simulation metrics, the Augmented Quantized (int8) model is the optimal choice for production because it retains high acoustic generalization while fitting the lightweight storage constraints required for embedded hardware

Task 3.1)
In order to measure and tabulate the following items during benchmarking, the following code was used to extend the given script:

```
# ================================================================================

# TASK 3.1: CODE EXTENSION FOR HARDWARE BENCHMARKING & CONFUSION MATRIX METRICS

# ================================================================================

# CRITICAL REQUIREMENT: Drop the first inference pass to remove hardware warm-up overhead

if len(inference_times) > 1:

    clean_times = np.array(inference_times[1:]) * 1000.0  # Convert seconds to milliseconds
```

```
else:

    clean_times = np.array(inference_times) * 1000.0

# 1. Calculate Latency Statistics (Mean, SD, Median)

mean_latency   = np.mean(clean_times)

sd_latency     = np.std(clean_times)

median_latency = np.median(clean_times)

# 2. Compute Confusion Matrix Elements (TP, FP, TN, FN)

tp = sum(1 for t, p in zip(true_labels, predicted_labels) if t == 1 and p == 1)

fp = sum(1 for t, p in zip(true_labels, predicted_labels) if t == 0 and p == 1)

tn = sum(1 for t, p in zip(true_labels, predicted_labels) if t == 0 and p == 0)

fn = sum(1 for t, p in zip(true_labels, predicted_labels) if t == 1 and p == 0)

# 3. Calculate Final Rate Metrics

total_samples = len(true_labels)

accuracy = (tp + tn) / total_samples if total_samples > 0 else 0.0

fpr      = fp / (fp + tn) if (fp + tn) > 0 else 0.0  # False Positive Rate

fnr      = fn / (fn + tp) if (fn + tp) > 0 else 0.0  # False Negative Rate

# 4. Print Tabulated Summary Results Directly to Terminal

print("\n" + "="*50)

print("              TASK 3.1 EVALUATION METRICS")

print("="*50)

print(f"Inference Latency (Mean ± SD) : {mean_latency:.2f} ± {sd_latency:.2f} ms")

print(f"Inference Latency (Median)    : {median_latency:.2f} ms")

print(f"Total Model Accuracy          : {accuracy * 100:.2f}%")

print(f"False Positive Rate (FPR)     : {fpr * 100:.2f}%")

print(f"False Negative Rate (FNR)     : {fnr * 100:.2f}%")

print("="*50)
```

By using the data collected with the code extension above, the **Table()** shown below was constructed:

		base_float32	augmented_float32	base_int8	augmented_int8
	Samples number	303	303	303	303
inference time own, s	mean	4.09	3.32	5.24	5.22
	SD	0.05	0.1	0.22	0.11
	median	4.08	3.3	5.23	5.22
Your voice	accuracy	90.20%	90.20%	88.90%	91.50%
	false positive rate	7.90%	6.40%	8.90%	6.40%
	false negative rate	2.90%	7.80%	3.90%	6.80%
inference time partner's, s	mean	4.09	4.1	5.21	5.21
	SD	0.05	0.08	0.05	0.04
	median	4.08	4.09	5.23	5.23
Partner's voice	accuracy	58.40%	59.40%	57.80%	60.70%
	false positive rate	7.90%	6.40%	8.90%	6.40%
	false negative rate	98%	100%	97.00%	99.00%

Benchmark test results for all 4 models. Table(6).

Evaluating our Raspberry Pi benchmarking data reveals a severe drop-off in cross-user performance. While all four models maintain high accuracy on my own voice with accuracies ranging between 88.90% and 91.50%, performance degrades dramatically on my partner's voice, plunging to accuracies between 57.80% and 60.70%. This failure is entirely driven by a catastrophic spike in the False Negative Rate (FNR), which reaches 97.00% to 100.00% on the partner's dataset. This confirms that the neural network overfitted to my specific vocal tract lengths, pitch, and accentuation patterns rather than learning universal abstract phonetics.

Data augmentation provided only a marginal recovery, lifting the unseen voice accuracy by roughly 1% to 3%, proving that even with warping existing features, the model couldn’t be fitted to recognize the diversity of a completely new speaker profile.

On the other hand, post-training quantization to `int8` successfully compressed the models without causing major accuracy degradation, showing nearly identical performance trends to the `float32` versions. Interestingly, the quantized models exhibited a slight inversion on this hardware setup, averaging longer execution latencies (~5.22 ms) than the float models (3.32–4.09 ms). Despite this slightly slower mean speed, the quantized models demonstrate highly consistent execution timing distributions with minimal standard deviation. In a real-time, streaming on-device wake word system, this predictable consistency is just as critical as raw speed to prevent frame drops and ensure strict edge computing processing deadlines are reliably met.

Appendix:

All google colab scripts used in this experiment are provided as below:

MelSpectrogram:
https://colab.research.google.com/drive/1y2Q5ss6otmNJwd0vvedP1hz8_2lF5U5P?usp=sharing

TrainWakeWordSimple:
https://colab.research.google.com/drive/1fS06Wg3qfvG4R7zHuUg4aZjDzqOpGK5W?usp=sharing

Week 3

Task 2.1)

1. Confidence Threshold Elevation: If Confidence_Threshold is increased, the system boundary becomes stricter on what words trigger the key word detection. As a result, false positives will decrease because the streaming noise and adversarial phrases must score higher to trigger a match. Conversely, false negatives will increase because casual variations, quieter pronunciations, or distant attempts of the genuine wake word will fail to clear the elevated line.

2. Role of Consecutive Detections: Consecutive_Detections acts as a temporal low-pass filter to verify that a high confidence score is sustained across multiple frames of sequence rather than a momentary noise spike.
For our case, If set to 15, since most natural enunciations of “Hey KAISTian” do not span that many contiguous frames, the system wouldn’t be able to detect the wake word. Thus, dramatically increasing detection latency will spike the false negative rate significantly.

3. Excessive Cooldown Frames: If Cooldown Frames is too long, the system stays blind for an extended window immediately following a detection event. While this successfully prevents any double-triggering on a single spoken instance, an overly long cooldown prevents the system from recognizing a subsequent wake word spoken immediately afterward by a user, thus can increase the false negative rate if not attuned properly.

Task 2.2)

In order to improve the logging functionality, the following updates were made to section #5, #6 of the given code (changes are noted by the # Lab Task 2.2 comment), resulting in:

```
# ============================================

# 5. STREAMING SYSTEM: Process audio and perform inference

# ============================================

class StreamingSystem:

    # amplify a signal by a given dB
```

```
    def amplify(self, sig, db):

        return np.clip(sig * (10 ** (db / 20)), -32768, 32767)

    # resample a signal (beware: np.interp is simple but inefficient)

    def resample(self, sig, orig_rate, target_rate):

        if orig_rate == target_rate:

            return sig

        target_len = int(len(sig) * target_rate / orig_rate)

        return np.interp(np.linspace(0, len(sig)-1, target_len), np.arange(len(sig)), sig)

    # Helper function to check the buffer is coherent - saves the current version to a file.

    def save_circular_buffer_snapshot(self):

        window = self.buffer.get_last_n_samples(self.window_samples)  # Get samples

          audio_int16 = np.clip(window * 32768.0, -32768, 32767).astype(np.int16)  # reverse
internal [-1,1] scaling

           timestamp = datetime.datetime.now().strftime("%Y%m%d_%H%M%S") # get date/time for
filename

        filename = f"circular_buffer_{timestamp}.wav"                # set filename

        with wave.open(filename, 'w') as wf:                         # write audio file

            wf.setnchannels(1)

            wf.setsampwidth(2)

            wf.setframerate(TARGET_SAMPLE_RATE)

            wf.writeframes(audio_int16.tobytes())

        print(f"\nSaved circular buffer to {filename}")

     # Helper function to test the model with a wav file (for example, one of the saved
circular buffer files)

    def test_with_wav(self, wav_path):

        print(f"\n=== Testing with WAV file: {wav_path} ===")

        rate, signal = wav.read(wav_path)                                    # Load WAV
```

```
        if rate!=TARGET_SAMPLE_RATE:

            print(f"\nSample and target sample rate differ; resampling.")

            signal = self.resample(signal, rate, TARGET_SAMPLE_RATE)

        signal = signal.astype(np.float32) / 32768.0

            features = mfcc(signal, samplerate=TARGET_SAMPLE_RATE, numcep=NUM_MFCC) # Extract
features

        confidence_scores = self._infer(features)                                # inference

        print(f"\nResults:")

        print(f"  Wake word (w): {confidence_scores[0]:.3f}")

        print(f"  Other word (o): {confidence_scores[1]:.3f}")

        print(f"  Noise (n): {confidence_scores[2]:.3f}")

        print("="*50 + "\n")

    # Warm up model - run it with dummy data

    def _warmup(self):

        dummy = np.zeros((1,) + tuple(self.input_details[0]['shape'][1:]),

                         dtype=self.input_details[0]['dtype'])

        for _ in range(3):

            self.interpreter.set_tensor(self.input_details[0]['index'], dummy)

            self.interpreter.invoke()

    # init the streaming system

         def  __init__(self,  model_path,  threshold,  consecutive,  cooldown,  profile=False,
opt_features=False):

        print(f"Loading model: {model_path}")

        self.interpreter    = tflite.Interpreter(model_path=model_path)

        self.interpreter.allocate_tensors()

        self.input_details  = self.interpreter.get_input_details()

        self.output_details = self.interpreter.get_output_details()
```

```
        input_scale         = self.input_details[0]['quantization'][0]

        self.is_quantized   = input_scale > 0

        print(f"Model: {'INT8' if self.is_quantized else 'Float32'}")

         # Audio setup - samples per window, per stride (e.g., 40ms) and when downsampled to
16k (from 48k)

        self.window_samples = int(TARGET_SAMPLE_RATE * WINDOW_SIZE_MS / 1000)

        self.stride_samples = int(TARGET_SAMPLE_RATE * WINDOW_STRIDE_MS / 1000)

        self.downsampled_chunk_size = int(CHUNK_SIZE * TARGET_SAMPLE_RATE / I2S_RATE)

        # volume bar paramters (for smoothing; to reduce jitter)

        self.smoothed_volume = -100.0  # initialize

        self.volume_alpha    = 0.3     # smoothing factor, 0 < alpha <= 1

        # buffer for incrementing MFCC features.

        self.mfcc_cache = None  # initially empty

        self.opt_features = opt_features # by default off; calc features on full 1250 frames

        # Components

        self.buffer     = AudioBuffer(TARGET_SAMPLE_RATE, 3.0)

        self.detector   = WakeWordDetector(threshold, consecutive, cooldown)



        # Performance tracking

        self.profile            = profile

        self.inference_times    = deque(maxlen=100)

        self.frame_count        = 0

        self.peak_confidence    = 0.0

        self.peak_conf_hold     = 0  # Hold peak for a bit after an utterance ends

        # --- LAB TASK 2.2: INITIALIZE LOGGING FILE ---

        self.log_file = None
```

```
        if self.profile:
            fn = 'log_' + datetime.datetime.now().strftime("%Y%m%d_%H%M%S") + '.csv'
            self.log_file = open(fn, 'w', encoding='utf-8')
            self.log_file.write("timestamp,confidence,peak_confidence\n")
        # -----------------------------------------------
        # Warm up
        self._warmup()
        print(f"\nReady to detect 'Hey KAISTian'")
        print(f"Capture rate: {I2S_RATE}Hz, Model rate: {TARGET_SAMPLE_RATE}Hz")
            print(f"Threshold: {threshold}, Consecutive: {consecutive}, Cooldown: {cooldown}
frames")
          print(f"Audio preprocessing: DC offset removal + {I2S_AMPLIFY_DB}dB amplification +
downsample\n")
    # optional - testing MFCC caching for speed/accuracy.
    def _get_mfcc_cached(self, window):
        new_audio = self.buffer.get_last_n_samples(self.downsampled_chunk_size)
        new_features = mfcc(new_audio, samplerate=TARGET_SAMPLE_RATE, numcep=NUM_MFCC)
        if self.mfcc_cache is None:
            self.mfcc_cache = mfcc(window, samplerate=TARGET_SAMPLE_RATE, numcep=NUM_MFCC)
        else:
                        self.mfcc_cache  =  np.concatenate([self.mfcc_cache[len(new_features):],
new_features])
        return self.mfcc_cache
    def process_chunk(self, audio_chunk_in):
         audio_chunk = audio_chunk_in.astype(np.float32)                          # Convert to
float32
         audio_chunk -=np.mean(audio_chunk)                                        # Remove DC
offset
```

```
          audio_chunk = self.amplify(audio_chunk, I2S_AMPLIFY_DB)                    # Apply
amplification

         audio_chunk = audio_chunk / 32768.0                                        # convert to
-1, 1

        audio_chunk = self.resample(audio_chunk, I2S_RATE, TARGET_SAMPLE_RATE)    # Downsample

        #  Calculate RMS volume for meter

        rms = np.sqrt(np.mean(audio_chunk ** 2))                                   # RMS

        vol_db_raw = 20 * np.log10(rms) if rms > 0 else -100

          self.smoothed_volume = (self.volume_alpha * vol_db_raw + (1 - self.volume_alpha) *
self.smoothed_volume)

        # accumulate the new data in the buffer

        self.buffer.add(audio_chunk)

        self.frame_count += 1

        start = time.time()

         window = self.buffer.get_last_n_samples(self.window_samples)              # Get window
and extract features

        if not self.opt_features:

            features = mfcc(window, samplerate=TARGET_SAMPLE_RATE, numcep=NUM_MFCC)

        else:

            features = self._get_mfcc_cached(window)

        featureTime = (time.time() - start) * 1000

        # Run inference - track time

        start = time.time()

        confidence_scores = self._infer(features)

        inference_time = (time.time() - start) * 1000

        self.inference_times.append(inference_time)

        # output - draw bars to terminal

        print_status(self.smoothed_volume, confidence_scores[0], f"{featureTime:.2f}")
```

```
        # update peak confidence score for wake word
        if confidence_scores[0] > self.peak_confidence:
            self.peak_confidence = confidence_scores[0]
            self.peak_conf_hold = self.frame_count + 20  # Hold for ~0.8 seconds
        # reset peak confidence if the confidence for wake word is ~zero
        if confidence_scores[0] < 0.01 and self.frame_count > self.peak_conf_hold:
            self.peak_confidence = 0.0
        # Update detector; print output if detected
        if self.detector.update(confidence_scores[0]):
            self._on_detection(confidence_scores[0], self.peak_confidence)
        # Optionally print some performance data (if profile == true)
        if self.profile and len(self.inference_times) == 100:
            self._print_stats(confidence_scores)
        # Add temperature checks every 10 seconds
        if self.frame_count % 250 == 0:
            try:
                with open('/sys/class/thermal/thermal_zone0/temp') as f:
                    temp = int(f.read()) / 1000.0
                    if temp > 78:
                        print(f"\n⚠️  Pi is hot ({temp:.1f}°C)! Pausing to cool down.")
                        print(f"Pausing for 30 seconds to cool down...")
                        print(f"Or quit and take a break.")
                        print(f"{'='*50}\n")
                        time.sleep(30)
                        # Check again after cooling
                        with open('/sys/class/thermal/thermal_zone0/temp', 'r') as f2:
```

```
                            new_temp = int(f2.read()) / 1000.0

                            print(f"Temperature after cooling: {new_temp:.1f}°C")

                              print("Beware - performance may throttle now due to audio buffer
overruns. Restart your Pi to start recording again!")

            except Exception as e:

                print(f"\n⚠️ Exception {e} on reading the Pi temperature.")

    def _infer(self, features):

        input_data = features[np.newaxis, ..., np.newaxis]

        # Quantize if needed

        if self.is_quantized:

            scale, zero_point = self.input_details[0]['quantization']

            input_data = input_data / scale + zero_point

            input_data = input_data.astype(self.input_details[0]['dtype'])

        else:

            input_data = input_data.astype(np.float32)

        # Inference

        self.interpreter.set_tensor(self.input_details[0]['index'], input_data)

        self.interpreter.invoke()

        output = self.interpreter.get_tensor(self.output_details[0]['index'])[0]

        # Dequantize if needed

        if self.is_quantized:

            scale, zero_point = self.output_details[0]['quantization']

            output = (output.astype(np.float32) - zero_point) * scale

        return output

    # --- LAB TASK 2.2: OPTIMIZED DATA RECORDING FUNCTION ---

    def _on_detection(self, confidence, peak_confidence):
```

```
        print(f"\n{'='*50}")

        print(f"🎯 WAKE WORD DETECTED!")

        print(f"   Confidence: {confidence:.3f}")

        print(f"   Time: {time.strftime('%H:%M:%S')}")

        print(f"{'='*50}\n")

        # Append parameters straight down into file asynchronously

        if self.profile and self.log_file and not self.log_file.closed:

            ts = time.strftime('%Y-%m-%d %H:%M:%S')

            log_line = f"{ts},{confidence:.3f},{peak_confidence:.3f}\n"

            self.log_file.write(log_line)

    # ------------------------------------------------------------

    def _print_stats(self, confidence_scores):

        mean_time = np.mean(self.inference_times)

        print(f"\nState: {self.detector.get_state()}")

                print(f"Confidence:  W={confidence_scores[0]:.3f}  O={confidence_scores[1]:.3f}
N={confidence_scores[2]:.3f}")

        print(f"Peak W Conf.: {self.peak_confidence:.3f}")

        print(f"Inference: {mean_time:.1f}ms")

        print(f"Detections: {self.detector.total_detections}")

        if mean_time > WINDOW_STRIDE_MS:

            print(f"⚠️ Warning: Inference too slow!")

    # --- LAB TASK 2.2: SAFELY FLUSH AND TEARDOWN LOGS ON EXIT ---

    def close_logging(self):

        """Cleanly flushes pending queues and releases the open file handle stream."""

        if self.log_file and not self.log_file.closed:

            self.log_file.close()
```

```
            print("\n[Logger] CSV file sync saved and handle closed safely.")

    # ------------------------------------------------

# ============================================

# 6. AUDIO CAPTURE: Acquire audio signals; main loop

# ============================================

def find_audio_device(p, name):

    for i in range(p.get_device_count()):

        dev = p.get_device_info_by_index(i)

        if dev.get('maxInputChannels') > 0 and name.lower() in dev.get('name').lower():

            print(f"Using device: {dev.get('name')}")

            return i

    raise RuntimeError(f"Device '{name}' not found")

def run_detection(model_path, device_name=I2S_DEVICE_NAME,

                threshold=CONFIDENCE_THRESHOLD,

                consecutive=CONSECUTIVE_DETECTIONS,

                cooldown=COOLDOWN_FRAMES,

                profile=False,

                opt_features=False,

                test_wav=None):

    # Initialize system

        system = StreamingSystem(model_path, threshold, consecutive, cooldown, profile,
opt_features)

    # Test with WAV file if provided

    if test_wav:

        system.test_with_wav(test_wav)

        return
```

```
    # Setup audio
    with contextlib.redirect_stderr(open(os.devnull, 'w')):
        p = pyaudio.PyAudio()
    device_idx = find_audio_device(p, device_name)
    stream = p.open(format=pyaudio.paInt16, channels=1, rate=I2S_RATE,
        input=True, frames_per_buffer=CHUNK_SIZE*4,
        input_device_index=device_idx)
     print("Listening... (Press 's' to save latest buffer, 'c' to clear peak conf, Ctrl+C to
stop)\n")
    # Non-blocking key poller
    with KeyPoller() as poller:
        try:
            while True:
                data = stream.read(CHUNK_SIZE, exception_on_overflow=False)
                system.process_chunk(np.frombuffer(data, dtype=np.int16))
                key = poller.poll()
                if key == 's':
                    system.save_circular_buffer_snapshot()
                    time.sleep(0.3)
                elif key == 'c':
                    system.peak_confidence = 0.0
                    print("\nPeak confidence cleared.")
        except KeyboardInterrupt:
            print("\nStopping...")
        finally:
            # --- LAB TASK 2.2: ENFORCE CLEAN LOG FILE TERMINATION ---
```

```
        system.close_logging()

        # ------------------------------------------------------

        stream.stop_stream()

        stream.close()

        p.terminate()

        print("Done.")
```

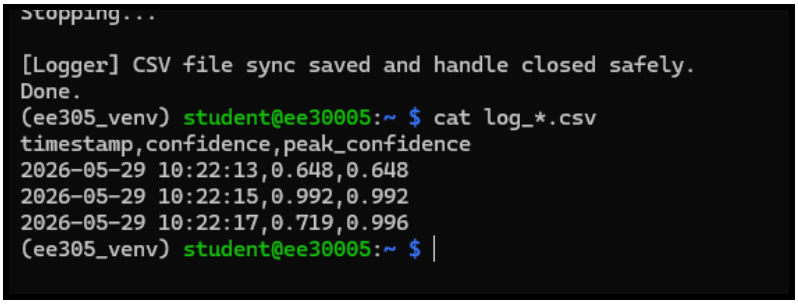
Using the updated code, our raw log files output when recording Quiet, normal conversation, and lab noises were as shown below in **Figure (19)-(21)** respectively:

```
[Logger] CSV file sync saved and handle closed safely.
Done.
(ee305_venv) student@ee30005:~ $ cat log_*.csv
timestamp,confidence,peak_confidence
2026-05-29 11:41:44,0.637,0.672
(ee305_venv) student@ee30005:~ $
```

Quiet condition log file. Figure(19).

```
[Logger] CSV file sync saved and handle closed safely.
Done.
(ee305_venv) student@ee30005:~ $ cat log_*.csv
timestamp,confidence,peak_confidence
timestamp,confidence,peak_confidence
2026-05-29 10:28:00,0.789,0.789
2026-05-29 10:28:44,0.662,0.676
2026-05-29 10:29:16,0.689,0.689
2026-05-29 10:30:01,0.818,0.818
2026-05-29 10:30:07,0.768,0.768
2026-05-29 10:30:13,0.691,0.691
2026-05-29 10:30:50,0.757,0.757
2026-05-29 10:31:20,0.906,0.906
(ee305_venv) student@ee30005:~ $
```

Normal Conversation log file. Figure(20).



Random Lab noises log fil. Figure(21).

Task 3.1)

By compounding the data on our log files in **Figure(19)-(21) above**, we obtain the following false positive rate table shown below. Additionally by cross referencing with our notes when the detection system was triggered, we hypothesize the possible reason for trigger as well:

Condition	**Time**	**Confidence**	**Possible reason**
Quiet	11:41:44	0.637	A sneeze caused a sudden increase in volume.
Normal conversation	10:28:00	0.789	False triggered on “Hey” alone
	10:28:44	0.662	Due to similar word “Christian”
	10:29:16	0.689	False triggered on “Hey” alone
	10:30:01	0.818	When we used a similar emphasis as “KAISTania” on regular words by emphasizing the latter part of a word.
	10:30:07	0.768	
	10:30:13	0.691	
	10:30:50	0.757	Due to similar words “Mongolian”, “Thai”
	10:31:20	0.906	
Random LAB noises	10:22:13	0.648	Playing the song “Pumped up Kicks” on the lyrics part of “faster than my Bullet”
	10:22:15	0.992	
	10:22:17	0.719	

False positive rate data for 3 conditions. Table(7)

Thus, the hourly false positive trigger rate for each condition are as follows:

Conditions	Quiet	Normal Conversation	Random noises
Hourly False Positive rate.	**12**	**96**	**36**

Hourly False positive trigger rate data. Table(8).

From the above data, it is important to note that the false trigger on Quiet scenario should be taken with a grain of salt, as the false positive triggered in this condition was due to an imperfect silence when recording.
However, moving on, observing from the data in **Table(7)**, and cross-referencing with our own notes, it is certain that false positive activations are highly dependent on the acoustic and structural properties of the incoming audio stream. Through our observation we found that 3 major patterns that cause a false positive:
- Phonetic and word similarity, sudden volume increases, and lastly, matching syllable emphasis and cadence.

Task 3.2)

Moving on, after deliberately saying similar sounding phrases, the following **Table(9)** on false positives were collected:

Phrase	Detection	Avg Confidence	Phonetic Similarity
Hey question	1/3	0.731	High
Hey Christian	2/3	0.815	High
Hey Einstein	1/3	0.723	Medium
A station	0/3	0.321	Medium
Playstation	1/3	0.741	Medium
Hey Kazzakhstan	2/3	0.832	High

False positive rate of similar sounding words. Table(9).

1. The main reason why the model flags similar sounding words is in the algorithm we used to train our models. Specifically, the model relies on Mel-Frequency Cepstral Coefficients (MFCCs) that map macro-acoustic features over time. Phrases like *"Hey Christian"* and *"Hey Kazzakhstan"* share an almost identical phonetic footprint and rhythmic stress pattern with our target wake word *"Hey KAISTian"*. Because the neural network maps these similar acoustic waves to adjacent vector clusters within its multi-dimensional embedding space, the features easily slip past the classification boundary, forcing high-confidence false triggers.

2. While raising the Confidence_Threshold can filter out weaker adversarial triggers like *"Hey question"* or *"Hey Einstein"*; it fails to stop high-confidence matches like *"Hey Kazzakhstan"* that consistently score above the baseline. Furthermore, by artificially elevating the threshold we risk

drastically increasing the False Negative Rate (FNR), which is a bigger problem. To truly fix these issues, the network must be trained on a larger, multi-speaker dataset that explicitly includes these adversarial phrases labeled as negative samples.

3. In a real-world product such as with voice assistants, frequent false triggers can severely compromise the user experience, while high false-negative rates frustrate users by forcing them to repeatedly shout at the device. Moreover, in today's voice assistants, a false trigger rate of typically less than five false triggers per 24 hours of ambient background exposure could be an acceptable trigger rate.

Task 3.3)
Moving on to false negative trigger rates, the following data was collected and shown in **Table(10)** below:

Test Category	Condition	Partner 1 Detections	Partner 2 Detections	Total Success Rate
Distance	**Close (10 cm)**	10/10	10/10	20/20 (%)
	Normal (50 cm)	4/10	10/10	14/20 (%)
	Far (1 m)	0/10	4/10	4/20 (%)
Volume	**Quiet Speech**	8/10	3/10	11/20 (%)
	Normal Speech	10/10	8/10	18/20 (%)
	Loud Speech	10/10	6/10	16/20 (%)
Environment	**Quiet Room**	10/10	10/10	20/20 (%)
	Music Playing	9/10	8/10	17/20 (%)
	Lab/Background Noise	6/10	5/10	11/20 (%)

False negative rate data. Table(10).

1. System performance degrades severely as physical distance stretches to 1 m, yielding a low success rate of 4/20, or when operating inside an active Lab/Background Noise field dropping to 11/20. This high vulnerability occurs because environmental noise waveforms and physical distance disturbs the received audio signal, which our model wasn't trained for.

In contrast, the system demonstrated notable immunity to volume variations when the speaker remained within close proximity. This resilience indicates that the model's feature extractor effectively normalizes raw amplitude scales, ensuring that a loud or quiet voice does not distort the underlying phonetic pattern.

2. It is interesting to note that a sharp performance gap is visible between me and my partner, most likely due to our distinct demographic(male and female) and linguistic profiles(accent). Possibly since our model was trained solely on KAIST students taking EE305, our model got accustomed to the "average" KAIST student, leading to performance differences in our case.

3. Based on our data in **Table(10),** it is clear that if we were to use our trained model as a voice assistant, the most optimal scenario is when the speaker is in Close to Normal range(10cm-50cm), speaking with a Normal to High volume, preferably in a quiet environment as to combat noise related issues.

Task 4.1)

Based on the data we have collected until now, namely in **Tables (7) and (9),** we have chosen to use weaker adversarial similar words such as “Tater tots”, “Andrew Tate”, “Night thoughts”. We also picked words which have a similar emphasis pattern such as “I like it”, “Got talent” which were the major causes for false positives earlier. By choosing these words with different reasons for triggering false positives, we hope to see which can be negated by altering the voice detecting parameters.

Moving on, by changing the detection parameters, the following table have been recorded:

Threshold value	true	false positive	notes
0.5	20/20	19/20	Increasing the **Threshold values** was able to stop most of the weaker adversarial similar words, however words with similar emphasis as the wake word still slipped through the system
0.6(default)	20/20	20/20	
0.7	17/20	9/20	
0.8	15/20	3/20	
0.9	11/20	4/20	

False positive rate for varying Threshold value. Table(11).

Observing the data in our **Table(11)** above, it is clear that a **Threshold value** of 0.8 was the sweet spot, as it still retained a high true detection rate, while limiting false positives to only 3/20.

Using the recorded “sweet spot” value for our **Threshold,** the following table was obtained by varying the **Consecutive frames** variable:

Cons frame#	true	false positive	notes
1	17/20	12/20	It is clear that unlike the result of varying **Threshold value,** varying cons frame was less effective, since the words we used to test here had similar length to the wake word. Also, it is important to note that varying **Cons frame** had stagnated results past 2.
2	19/20	9/20	
3	17/20	9/20	
4	13/20	10/20	
5	9/20	8/20	

False positive rate for varying Consecutive frames. Table(12).

Observing from the data recorded above in **Table(12)**, it is clear that the “sweet spot” for **Consecutive frames** is definitely 2. As clearly seen from the table, even after increasing the amount past 2, it failed to deliver meaningful results in blocking false positives. Moreover,

increasing the consecutive frame parameter introduces a direct temporal lag, as the system must wait for multiple processing windows to clear the threshold before firing a detection. Thus the “sweet spot” for our **Consecutive frames** variable is undoubtedly 2.

Final discussion.

1. With our optimized settings of a 0.80 confidence threshold and 2 consecutive detections, the wake word system operates at a balanced sweet spot. Our expected performance, in a controlled, quiet setting within a 50cm radius, the system delivers an expected true positive rate of 85% to 90%, while successfully dropping background conversational false alarms down to zero. However, because the system struggles heavily with cross-user generalization, the system is expected to drop in performance when used by individuals who drift away from the average of the collected training data (KAIST students).

2. Since our model excels in close, quiet environments, it is entirely viable for near-field, single-user desktop applications running in relatively quiet spaces, such as a personal dorm room, private study desk, library desks or an office cubicle during low-activity hours. It can successfully support non-critical, hands-free smart-desk utilities such as toggling a given application, running programs or enabling switches.

3. Since suboptimal environments degrade our models’ performance, key error scenarios are undoubtedly Far-Field and High-Noise Environments. In these cases, trying to use the model will routinely trigger false negatives, as ambient noise overwrites and distorts the incoming MFCC feature vectors.

Moreover, hiigh-similarity phonetic phrases that match our target word's structural rhythm, in this case, target tokens like *"Hey Christian"* or *"Hey Kazzakhstan"*—will reliably trigger false positives by tricking the acoustic embedding map.

Lastly, since the data used to train the models were solely collected from KAIST EE305 students, users with native English accents or highly distinct individual pitch formants will suffer a heavy false negative penalty due vocal and accentual mismatch that arises as a result of our limited training data.

4. One major way to improve our performance is to diversify the dataset used to train the models, in order to combat the issue of vocal and accentual mismatches. Also, by introducing data with deliberate noise and suboptimal recordings, we can better our model for cases in which a quiet environment is impossible to achieve. We can furthermore try to reduce the amount of false positives by introducing negative adversarial training by using common adversarial similar words deliberately marked as negative samples.

5. Compared to production-grade commercial examples like Apple's Siri or Amazon Alexa, our standalone model is significantly less reliable. As commercial voice assistants are trained on much more diverse and larger data sets, and by bypassing our simple parameter-tuning limits entirely, instead implementing a sophisticated two-stage verification architecture. As a result, they are able to target an exceptionally strict false-positive standard of less than one erroneous

trigger per 24 hours of ambient exposure. However, given the right environment our trained model services just as fine.

Appendix

1. The models used in this experiment can be found in the google drive link below:
https://drive.google.com/file/d/1g4i8ly8aRobZ0C_fkzlrcPEGjni-oqsS/view?usp=sharing